📘 CLASS LEADER GUIDE

Pre-Class Checklist (do these the day before)

- [ ] Each attendee's PC has Arduino IDE installed
- [ ] Board package: **esp32 by Espressif Systems**
- [ ] These libraries via Library Manager:
  - Arduino_GFX_Library (moononournation)
  - ArduinoJson (Benoit Blanchon) v7.x
  - WiFiManager (tzapu) v2.0.x
  - SensorLib (Lewis He)
- [ ] All five sketch files in attendees' `FlightRadar/` folder (`.ino` + 4 headers)
- [ ] Tools settings written on whiteboard (see below)
- [ ] One assembled, working device for demo
- [x] Spare USB-C cables (you'll need them)
- [x] **Note your venue's WiFi password** — attendees will join their devices to it
- [ ] Cases printed/cut and ready

Tools menu settings (write on whiteboard)

```
Board ................. Wavesheare ESP32-S3-Touch-LCD-1.46
USB CDC On Boot ....... Enabled
Events Run On ......... Core 0
Flash Mode ............ QIO 120MHz
Arduino Code Runs On .. Core 1
Partition Scheme ...... 8M with spiffs (3MB APP/1.5MB SPIFFS)
PSRAM ................. Enabled
Upload Speed .......... 921600
USB Mode .............. Hardware CDC and JTAG
```

These exact settings are critical. PSRAM "Disabled" = blank screen.

Suggested Schedule (2 hours)

Time	Block	What's happening
0:00 – 0:10	Welcome + demo	Show the working radar. Point out features. Hand out handouts.
0:10 – 0:25	Hardware tour	Walk through the board components
0:25 – 0:55	Code walkthrough I	Architecture, board_config, settings, ADS-B fetch
0:55 – 1:00	Break	

Time	Block	What's happening
1:00 – 1:25	Code walkthrough II	Rendering, sweep math, touch state machine
1:25 – 1:45	Flash session	Follow Build Guide. Troubleshoot common issues.
1:45 – 2:00	Wrap, Q&A, case handout	Mod ideas, GitHub link, next steps

Adjust freely. If hands-on flashing reveals problems, eat into the wrap time.

Hardware Tour Talking Points

Pass an unmounted board around and trace these out:

1. **ESP32-S3 chip itself** — point out it's a *system on chip*: CPU + WiFi + Bluetooth + lots of GPIO in one package.
2. **The square IC labeled SPD2010** — show that it handles *both* the display (over QSPI, four data lines) and touch (over I²C, two data lines). Same chip, two interfaces, two roles.
3. **QMI8658** — the IMU. Three accelerometers + three gyros measuring how the board is moving and oriented. Pick the board up and tilt it to demonstrate.
4. **PCF85063** — RTC with a coin cell. Discuss why a separate RTC exists when the MCU could keep time (answer: power efficiency and survival across reboots/battery swaps).
5. **PCM5101** — dedicated audio DAC. Why? Better sound than PWM, and offloads work from the CPU.
6. **TCA9554PWR** — I/O expander. Discussion: when you run out of MCU pins, you put another chip on the I²C bus that gives you more pins. We use it for reset lines and the SD card chip select.
7. **8 MB PSRAM** stacked on the ESP32-S3 — this is what holds the 340 KB display framebuffer.

Code Walkthrough Talking Points

Block 1 — Architecture (5 min)

Open the sketch in Arduino IDE. Point out the **five files**:

- `FlightRadar.ino` — main logic in three parts
- `board_config.h` — *every pin definition* lives here. "If we wanted to support a different board, we'd add another `#elif` block and nothing else changes."
- `font_config.h — this is where we control font sizes
- `themes.h` — color palettes
- `aircraft_types.h` — ICAO code lookup

Key concept: *hardware abstraction*. Tomorrow if you swap to a different ESP32 board, you change one file.

Block 2 — The Settings struct + NVS (5 min)

Show `struct Settings` and `loadSettings()` / `saveSettings()`. Talk about:

- **NVS (Non-Volatile Storage)** — flash partition that survives reboots
- Key/value pairs by string name

- Sane defaults applied on first boot
- All persistent state lives in *one* struct — easy to reason about

Block 3 — The Aircraft struct + the mutex (10 min)

Show `struct Aircraft`, then jump to `aircraftMutex`.

Key concept: *concurrency*. Two cores (or tasks) cannot safely touch the same data simultaneously. The mutex is a "talking stick." Demonstrate: the ADS-B fetch task takes it to write new aircraft; the render code takes it to read. They never collide.

> "If we forgot the mutex, we'd see weird flickering blips or random crashes — but not consistently, which is the worst kind of bug."

Block 4 — The ADS-B fetch task (10 min)

Walk through `fetchOnce()` → `parseAdsbResponse()` → `handleAdsbResults()`.

Discussion points:

- Why a **separate FreeRTOS task on Core 0**? — HTTP can take seconds; the UI on Core 1 can't afford to freeze.
- ArduinoJson v7's **JsonDocument** auto-allocates from PSRAM
- Walk through one aircraft: hex, callsign trimming, type lookup, altitude, category mapping
- **The haversine formula** in `computeRelative()` — great-circle distance over a sphere. A spot to pause and let people google if they want.

Block 5 — Coordinate Math + Sweep (10 min)

Open `polarToScreen()`. This is where everyone says "ohhh."

Whiteboard sketch: a circle with bearing 0° at top, going clockwise. Show the conversion `x = CX + r·sin(θ)`, `y = CY − r·cos(θ)`. Note that screen Y grows *downward*, hence the minus sign.

Then `advanceSweep()`:

- The sweep rotates ~60°/sec.
- For each aircraft, we compute the angular distance behind the sweep line.
- Just-passed-over = bright (255). The further behind, the dimmer (down to 35).
- This is the **phosphor afterglow** effect from real CRT radar.

Block 6 — Touch State Machine (10 min)

This is the trickiest piece. Set expectations: *"This was hard to get right."*

Open `pollTouchAndHandleGestures()`. Talk through the states:

- `TS_IDLE` → finger lands → `TS_DOWN`
- Big enough movement → `TS_DRAGGING`

- Outer edge touch → `TS_EDGE_BRIGHT`
- On release: was it a swipe (>60 px in one axis)? If not, treat as tap.

Key lesson: raw input is messy. You have to convert a *stream of samples* into a *gesture*. A state machine is the standard way.

Block 7 — The SPD2010 touch driver (10 min, optional advanced)

For groups that want to go deeper, open `readTouch()`. Talk through the four-step state machine:

1. Read status
2. Issue commands to get the chip into point-reporting mode
3. Read the HDP packet of touch coordinates
4. Acknowledge

> "Most touch chips are simple register reads. This one's a small protocol. That's why our first attempt didn't work — we treated it like a simpler chip."

A nice teaching moment about *reading datasheets and vendor code* when libraries don't help.

Flash Session

Everyone opens the sketch, verifies Tools settings (have them check off the whiteboard list), and clicks Upload.

Common issues and fixes (announce these proactively)

Error	Fix
`'TouchPoint' was not declared`	They edited `board_config.h` — confirm `struct TouchPoint` is still defined at the bottom
`Arduino_SPD2010` constructor error	Library out of date — update Arduino_GFX_Library
Boot loop on upload	Press and hold BOOT button while clicking Upload, release after "Connecting…" appears
Blank screen after upload	PSRAM not set to OPI
Can't see flightradar.local	Some Windows networks block mDNS — use the IP shown in the serial monitor

Recovery procedure

If a device is wedged, hold BOOT, click Upload, release on "Connecting…". Always works.

Discussion Prompts (for breaks or wind-down)

Open-ended things to ask:

- *"What would you change about the UI?"*
- *"What would it take to support a second board model?"* (Answer: a new block in `board_config.h`)
- *"How would you add aircraft trails — the path a plane has flown?"* (Hint: stash recent positions per aircraft, draw fading line behind)
- *"The ADS-B fetch costs about how many bytes per request?"* (Order of magnitude estimate using their data plan)
- *"Why is the framebuffer 340 KB and where does it live?"* (412×412×2 bytes in PSRAM)

Extension Activities (for fast finishers or homework)

Difficulty	Activity
⭐	Change a theme color and reflash
⭐	Add a new range preset (e.g., 100 mi)
⭐⭐	Add a third theme (e.g., "Blue Vector")
⭐⭐	Add 10 new aircraft type codes from the FAA registry
⭐⭐⭐	Show flight trails — keep last 5 positions per aircraft
⭐⭐⭐	Add a web endpoint that returns current aircraft as JSON
⭐⭐⭐⭐	Replace the speaker chirps with synthesized tones using sine waves
⭐⭐⭐⭐	Add a "Heat map" mode showing where flights cluster over time

After-Class Cleanup

- Collect any uncovered boards, cables
- Note any feedback for next iteration
- Push code updates to the class GitHub
- Email a recap with the GitHub link and the handout PDF

A Note on Where Help Is Buried in This Project

If something breaks for an attendee outside of class, the most useful pointers (in order):

1. **Serial Monitor at 115200 baud** — every init step prints success/failure. `[Boot] ready` is what they want to see.
2. **`board_config.h`** — 90% of "weird hardware behavior" is a pin define disagreeing with reality.
3. **`canvas->begin()` returning false** — usually PSRAM not enabled. Fix in Tools menu and reflash.
4. **The Arduino_GFX bundled example `PDQgraphicstest`** with `#define WAVESHARE_ESP32_S3_LCD_1_46` — if that draws correctly and yours doesn't, the problem is in our setup order, not the library.